OpenAI Codex, Work & Custom GPTs

Beginner End-to-End Notes

Detailed classwise guide for the updated 46-video playlist

EXPLAIN • PRACTISE • VERIFY

A practical study companion covering Codex workflows, skills, plugins, MCP, Custom GPTs, local models, agent harnesses, automation, observability, and multi-agent work.

How to use this guide

Each class contains a short overview, learning goals, detailed numbered explanations, and revision takeaways. Use the linked video whenever you want to replay the demonstration.

Contents

Class	Topic
1	From NotebookLM outputs to a custom video-creation plugin
2	Build a web app with a specialist plugin
3	MCP fundamentals and the integration problem it solves
4	Installing reusable skills from skills.sh
5	Skills versus plugins
6	Voice dictation with Whisper Flow and reusable assistants
7	Convert a Google Flow prompt into a reusable skill
8	Run the documentary-video skill end to end
9	Image-first media creation and browser-assisted production
10	Gemini APIs for speech, voice, and music workflows
11	Create and install your own Codex skill
12	Project instructions with AGENTS.md
13	Computer use, file creation, and publishing workflows
14	Codex Work and everyday desktop workflows
15	Building a full-stack application with an API and database

16	APIs and actions for Custom GPTs
17	Create, test, and publish a Custom GPT
18	Build a multimedia plugin with Plugin Creator
19	Media-plugin practice and an MCP bridge
20	MCP architecture through the restaurant analogy
21	Wrap an existing API as an MCP server
22	Build a movie-booking MCP server
23	Deploy an MCP-backed movie application
24	MCP configuration and connecting the server to Codex
25	Use the movie MCP workflow from another client
26	Build an OpenAI-powered chatbot application
27	Build an AI palm-reading app and a self-contained 3D game
28	Codex cloud tasks, uploaded skills, and scheduled work
29	Why run AI models locally?
30	Install and use Ollama
31	Run Ollama on a cloud GPU virtual machine
32	Cloud images, SSH keys, and secure VM access
33	Open WebUI, Docker, and remote Ollama access
34	Install Hermes Agent on Windows
35	Use different cloud and local models in Hermes Agent
36	Configure providers, endpoints, and skills in Hermes
37	Prompt, context, harness, and loop engineering

38	Connect agent harnesses to local or cloud models
39	Connect a Telegram bot to an AI agent
40	Schedule recurring agent jobs
41	Agent harnesses and complete project execution
42	Troubleshoot a harness and choose models by context limits
43	Observe LLM applications with Langfuse
44	Install and configure OpenClaw
45	Add a Gemini TTS skill to OpenClaw
46	Multi-agent orchestration and loop engineering

Class 1: From NotebookLM outputs to a custom video-creation plugin

Class video: Open on YouTube

This class starts with NotebookLM's study-media features and then explains why a reusable Codex plugin is useful when you need tighter control over video creation.

What you'll learn

- What NotebookLM can generate from source material.
- Why fixed-duration, repeatable production needs a custom workflow.
- How images, narration, and FFmpeg become one video pipeline.

Step-by-step notes

1. **Use NotebookLM for source-grounded learning assets.** After adding reliable sources, it can create audio overviews, slide decks, videos, mind maps, reports, flashcards, and infographics. Select the output language and style before generating.
2. **Recognize the limitation.** A one-click generated video may be useful, but it does not always obey an exact length, image count, or scene structure. Those requirements belong in a reproducible workflow.
3. **Break video creation into stages.** A practical pipeline is: plan scenes → generate one visual per scene → write narration → synthesize speech → combine images and audio → validate the final file.
4. **Package the workflow as a plugin.** The plugin can expose simple inputs such as topic, duration, language, and aspect ratio while hiding the repeated technical steps.
5. **Use FFmpeg for assembly.** FFmpeg joins scene media, normalizes formats, and produces a final MP4. Always check that the final duration, dimensions, and audio stream match the request.

Key takeaways

- NotebookLM is strong for fast, source-based learning media.
- A custom plugin is better for repeatable output with strict controls.
- Reliable video creation is a pipeline, not a single prompt.

Class 2: Build a web app with a specialist plugin

Class video: Open on YouTube

This class demonstrates how a web-development plugin gives Codex focused instructions and tools, using a student-marks calculator as the example project.

What you'll learn

- What a specialist plugin adds to a general AI assistant.
- How to turn a plain request into an implementable web specification.
- How to validate a small HTML, CSS, and JavaScript project.

Step-by-step notes

1. **Install or enable the web-app plugin.** The plugin contributes a workflow, quality rules, and implementation guidance for frontend projects.
2. **State the behavior precisely.** The demo asks for subject marks between 0 and 100, a pass rule of at least 35 in every subject, and calculated total and percentage.
3. **Request a clear file structure.** A beginner-friendly version uses `index.html`, `style.css`, and `script.js`. Keeping structure, styling, and behavior separate makes debugging easier.
4. **Validate before calculating.** Reject blank, non-numeric, negative, or over-100 values. Do not silently calculate invalid marks.
5. **Test boundary cases.** Check 34 versus 35, 0, 100, empty fields, and mixed pass/fail subjects. A visually polished app is not correct until these cases work.

Key takeaways

- Plugins add domain-specific process, not merely prettier output.
- A good prompt specifies inputs, rules, outputs, files, and tests.
- Test the business rule at its boundaries.

Class 3: MCP fundamentals and the integration problem it solves

Class video: Open on YouTube

This class introduces the Model Context Protocol (MCP) as a common way for AI applications to discover and use external capabilities.

What you'll learn

- Why point-to-point AI integrations do not scale well.
- The roles of an MCP host, client, and server.
- How MCP differs from a model or an ordinary API.

Step-by-step notes

1. **Start with the old problem.** If every AI model needs a separate custom connector for every application, the number of integrations grows quickly and each one must be maintained separately.
2. **Use a shared protocol.** MCP standardizes how tools, resources, and prompts are exposed to an AI application. It was introduced by Anthropic and released as an open standard.
3. **Identify the parts.** The **host** is the AI application, the **client** manages the connection inside the host, and the **server** exposes capabilities for a particular service or data source.
4. **Follow the request path.** User request → model decides a tool is needed → MCP client calls the appropriate server → server talks to the underlying application/API → result returns to the model.
5. **Keep trust boundaries clear.** An MCP server can expose powerful actions. Review its publisher, permissions, authentication method, and available tools before connecting it.

Key takeaways

- MCP is an integration standard, not an AI model.
- One well-designed MCP server can serve multiple compatible AI clients.
- Tool access should be treated as privileged access.

Class 4: Installing reusable skills from skills.sh

Class video: Open on YouTube

This class explains skills as portable instruction packages and shows how a published skill can be discovered, inspected, and installed.

What you'll learn

- What a `SKILL.md` file contains.
- How the skill installer helps discover and add skills.
- Why the same skill can be useful across different coding agents.

Step-by-step notes

1. **Think of a skill as a playbook.** It tells the agent when to use a workflow, what steps to follow, what files or scripts to reuse, and what quality checks are required.
2. **Inspect before installing.** On a skill directory such as skills.sh, read the description and source. Check whether it runs scripts, accesses external systems, or expects credentials.
3. **Install by name or repository URL.** The skill installer places the package where the agent can discover it. Restart or refresh the session when the host requires it.
4. **Trigger the skill with a matching request.** The demonstration uses a frontend-design skill so the agent applies design-specific guidance rather than producing a generic page.
5. **Verify the result.** Confirm the skill is listed, read its instructions, and run a small test task before relying on it for a larger project.

Key takeaways

- Skills package repeatable knowledge and process.
- Read third-party skill instructions before trusting them.
- A portable skill reduces repeated prompting across projects and agents.

Class 5: Skills versus plugins

Class video: Open on YouTube

This class separates two ideas that are often confused: a skill teaches an agent how to work, while a plugin can bundle skills and connect the agent to tools or services.

What you'll learn

- The practical difference between a skill and a plugin.
- When instructions alone are enough.
- When authentication, APIs, or external actions require an integration.

Step-by-step notes

1. **Skill = knowledge and workflow.** A skill can contain instructions, examples, templates, reference material, and scripts. It improves how the agent approaches a task.
2. **Plugin = packaged capability.** A plugin can bundle one or more skills and may also provide tools, MCP servers, configuration, or access to external applications.
3. **Use the brain-and-hands analogy.** The skill is the specialized know-how; the connected tool is the ability to act. A GitHub workflow guide can explain good practice, while a GitHub integration can actually read issues or create a pull request.
4. **Choose the smallest solution.** If the task only needs a repeatable writing or coding method, a skill may be sufficient. If it must send mail, update Slack, or query a live service, use an authorized integration.
5. **Review scope.** Installation does not mean every action is safe. Check what data the plugin can read and what state it can change.

Key takeaways

- Skills primarily guide reasoning and process.
- Plugins package broader capabilities and may perform external actions.
- Permission review matters whenever a capability reaches outside the workspace.

Class 6: Voice dictation with Whisper Flow and reusable assistants

Class video: Open on YouTube

This class explores voice-to-text as a faster input method and connects it to the idea of skills as reusable virtual assistants.

What you'll learn

- Where dictation improves day-to-day prompting.
- How to use a system shortcut to capture speech.
- Why voice input and task automation are separate concerns.

Step-by-step notes

1. **Use dictation for first drafts and long instructions.** Speaking is often faster for context, acceptance criteria, meeting notes, or brainstorming.
2. **Configure the correct shortcut.** The session demonstrates the Whisper Flow desktop workflow and its platform shortcut. Verify the current shortcut in the installed application's settings because it may differ by operating system or version.
3. **Review the transcription.** Correct names, command-line flags, punctuation, and file paths before submitting. Speech recognition can turn a small technical word into a materially different instruction.
4. **Separate capture from execution.** Dictation only enters text. A skill may structure the request, but the user still needs an appropriate trigger or workflow to run it.
5. **Create focused assistants.** Reusable skills can act like role-specific assistants for documentation, testing, content production, or project setup.

Key takeaways

- Voice input is best for speed; review is still required for precision.
- A shortcut does not replace a well-defined workflow.
- Narrow, reusable assistants are easier to trust and improve.

Class 7: Convert a Google Flow prompt into a reusable skill

Class video: Open on YouTube

This class takes a fragile manual video prompt and turns it into a repeatable browser-assisted production workflow.

What you'll learn

- How to identify repeatable steps in a manual process.
- How a skill can control prompt creation, downloads, and stitching.
- Why browser automation requires checkpoints.

Step-by-step notes

1. **Write down the manual flow first.** Create a project, select the video model, submit one scene prompt at a time, wait for generation, download each clip, and merge the clips.
2. **Convert choices into inputs.** Typical inputs are topic, number of clips, language, tone, and aspect ratio. The skill should derive scene prompts from these values.
3. **Make prompts scene-specific.** Each prompt should describe subject, action, environment, camera behavior, lighting, and continuity with adjacent scenes.
4. **Automate the browser carefully.** Browser control can navigate and enter prompts, but generation delays, pop-ups, and changed page layouts can interrupt the sequence. Verify state before the next action.
5. **Download and assemble deterministically.** Save clips with ordered names such as `scene-01.mp4`, then stitch them with FFmpeg and validate playback.

Key takeaways

- Automate a documented workflow, not an undefined idea.
- Stable naming and ordering prevent assembly errors.
- Browser automation needs state checks around slow or variable operations.

Class 8: Run the documentary-video skill end to end

Class video: Open on YouTube

This class uses the custom Flow skill to create a short documentary from a few high-level inputs and then validates the merged output.

What you'll learn

- How high-level inputs become a scene blueprint.
- How clips are generated and collected through Chrome.
- How to merge and quality-check the finished video.

Step-by-step notes

1. **Provide production constraints.** State the topic, desired number of clips or total duration, output language, aspect ratio, and visual tone.
2. **Review the scene blueprint.** Ensure the sequence has an opening hook, logical development, and a closing point. Remove repeated scenes before spending generation credits.
3. **Generate in order.** Submit one prompt, wait until the clip is complete, download it, and confirm the file before advancing.
4. **Merge with FFmpeg.** When all clips share compatible codecs and dimensions, use a concat list; otherwise re-encode to a common format before joining.
5. **Perform final checks.** Watch the complete video, verify scene order, listen for missing or clipped audio, and inspect resolution, aspect ratio, and total duration.

Key takeaways

- The scene plan is the quality foundation for the final video.
- Never assume a download succeeded merely because generation finished.
- Validate the final MP4, not just the individual clips.

Class 9: Image-first media creation and browser-assisted production

Class video: Open on YouTube

This class extends the media workflow by treating images as planned scene assets and using browser control to help create and organize them.

What you'll learn

- Why visual planning should happen before assembly.
- How to keep generated images consistent across scenes.
- How to organize assets so later automation is reliable.

Step-by-step notes

1. **Plan one visual purpose per scene.** Decide what the viewer must understand, then describe the subject, setting, composition, color, and camera angle.
2. **Lock the visual language.** Reuse character descriptions, palette, lighting, and aspect ratio. Uncontrolled variation makes the final video feel like unrelated stock images.
3. **Use the browser as an execution surface.** The agent can enter prompts and download outputs, but the user should confirm sign-in state and any credit-consuming action.
4. **Name assets predictably.** Use names such as `01-hook.png`, `02-problem.png`, and `03-solution.png`. This makes scene-to-file mapping obvious.
5. **Reject weak assets early.** Check text rendering, hands, faces, logos, aspect ratio, and subject consistency before narration and editing.

Key takeaways

- Image-first planning improves narrative continuity.
- Consistent prompts and filenames reduce rework.
- Quality control should happen before final assembly.

Class 10: Gemini APIs for speech, voice, and music workflows

Class video: Open on YouTube

This class introduces API-based media generation, with emphasis on text-to-speech and other audio outputs that can be reused in automated content pipelines.

What you'll learn

- The difference between using a web interface and calling an API.
- The basic inputs to a text-to-speech request.
- How to handle credentials and generated audio safely.

Step-by-step notes

1. **Create and protect an API key.** Store it in an environment variable or secret manager. Do not paste it into source files, screenshots, or public repositories.
2. **Choose the correct model and output type.** A TTS request needs text, voice or speaker settings, and an audio format. Music or song generation may use different models and constraints.
3. **Test with a short sample.** Confirm pronunciation, language, speaking rate, and output format before synthesizing the full script.
4. **Save audio with scene-aware names.** For example, `scene-03-narration.wav` clearly connects the audio to its visual.
5. **Normalize before editing.** Different providers can return different sample rates or loudness. Convert assets to a consistent format before combining them.

Key takeaways

- APIs make media generation repeatable and programmable.
- Keys belong in secrets, never in committed code.
- Short pronunciation tests save time and cost.

Class 11: Create and install your own Codex skill

Class video: Open on YouTube

This class moves from consuming skills to authoring one, with an emphasis on clear triggers, reusable instructions, and validation.

What you'll learn

- The essential structure of a custom skill.
- How to write instructions an agent can follow reliably.
- How to package, install, and test the skill.

Step-by-step notes

1. **Define one narrow job.** Write a clear description of when the skill should activate and what outcome it owns.
2. **Create `SKILL.md`.** Include the workflow, required inputs, safety rules, expected outputs, and completion checks. Avoid vague instructions such as “make it good.”
3. **Add reusable resources.** Put templates, reference documents, and deterministic helper scripts in the skill package rather than repeating them in every prompt.
4. **Install the skill.** Place or import it into the supported skills directory, then start a fresh task so discovery happens cleanly.
5. **Test positive and negative triggers.** Confirm that a matching request activates the skill and an unrelated request does not.

Key takeaways

- A skill should solve a repeatable, bounded problem.
- Explicit inputs and completion checks improve reliability.
- Trigger testing is part of skill quality.

Class 12: Project instructions with AGENTS.md

Class video: Open on YouTube

This class explains how repository-level instructions give Codex persistent context about a project's conventions and required workflows.

What you'll learn

- What belongs in `AGENTS.md`.
- How project instructions differ from a reusable skill.
- How scoped instructions reduce repeated corrections.

Step-by-step notes

1. **Use `AGENTS.md` for repository-specific rules.** Examples include build commands, test commands, directory ownership, naming conventions, and files that must not be changed.
2. **Keep instructions executable.** Prefer “run `npm test` before handoff” over broad statements such as “ensure quality.”
3. **Document project structure.** Tell the agent where application code, tests, documentation, generated assets, and configuration live.
4. **Reference skills when appropriate.** A reusable design or document skill can supply general expertise; `AGENTS.md` adds the local rules for this project.
5. **Update stale guidance.** Incorrect commands in an instruction file are worse than no command because the agent will repeatedly follow them.

Key takeaways

- `AGENTS.md` is local project policy.
- Skills are reusable across projects; project instructions are repository-specific.
- Concrete commands and boundaries are the most useful content.

Class 13: Computer use, file creation, and publishing workflows

Class video: Open on YouTube

This class shows Codex operating across files and applications, then emphasizes the checks needed before publishing the result.

What you'll learn

- How computer-use automation differs from normal code generation.
- How to combine UI actions with local file work.
- What to verify before making an artifact public.

Step-by-step notes

1. **Choose the correct execution method.** Use direct file or command-line tools when available; use computer control when the task genuinely depends on a graphical application.
2. **Work in observable stages.** Open the target, inspect current state, make one logical change, and verify it before continuing.
3. **Preserve user data.** Do not overwrite unrelated files or assume an open document is disposable. Save generated work to a clear output location.
4. **Preview the artifact.** Open the site, document, or app and test the core user path. A successful command does not guarantee a usable result.
5. **Publish deliberately.** Confirm destination, visibility, secrets, licensing, and final content before deployment or sharing.

Key takeaways

- UI automation is useful when an API or direct tool is unavailable.
- Small checkpoints make automation easier to recover.
- Publishing is a separate, higher-impact step that deserves review.

Class 14: Codex Work and everyday desktop workflows

Class video: Open on YouTube

This class positions Codex Work as an environment for longer, multi-step tasks that combine conversation, workspace files, and tools.

What you'll learn

- How to give Codex enough context for a work task.
- How to divide a large request into verifiable outcomes.
- How to review changes without reading every internal step.

Step-by-step notes

1. **State the outcome and scope.** Name the folder, artifact, or application to work on and describe what “done” means.
2. **Provide constraints early.** Mention required formats, technologies, reference examples, deadlines, and things that must remain unchanged.
3. **Let the workspace carry context.** Existing files, instructions, and previous outputs can be inspected so the agent does not rely only on the chat prompt.
4. **Ask for evidence.** Useful evidence includes test results, rendered previews, changed-file lists, or links to the final artifact.
5. **Review impact before external actions.** Sending, publishing, purchasing, or changing a live system should be distinguished from creating a draft locally.

Key takeaways

- Outcome, scope, and acceptance criteria make work requests effective.
- Workspace context reduces repetitive prompting.
- Verification should match the risk of the task.

Class 15: Building a full-stack application with an API and database

Class video: Open on YouTube

This class brings frontend, backend, and persistence together and shows why the data contract should be decided before implementation.

What you'll learn

- The responsibilities of frontend, API, and database layers.
- How a request travels through a full-stack application.
- What to test before calling the application complete.

Step-by-step notes

1. **Define the data model.** List entities, required fields, identifiers, and relationships. This becomes the foundation for database tables and API payloads.
2. **Design the API contract.** Specify endpoints, HTTP methods, request bodies, success responses, and error responses before wiring up the UI.
3. **Implement server-side validation.** Frontend checks improve usability, but the backend must enforce required fields, types, authorization, and business rules.
4. **Connect the user interface.** Show loading, success, empty, and failure states. Do not leave a failed network call as a silent button click.
5. **Test end to end.** Create, read, update, and delete real records; restart the application to confirm persistence; and test invalid and unauthorized requests.

Key takeaways

- Full-stack work depends on a stable contract between layers.
- Validation belongs on the server even if the browser also validates.
- An end-to-end test proves more than isolated screenshots.

Class 16: APIs and actions for Custom GPTs

Class video: Open on YouTube

This class explains how a Custom GPT can move beyond uploaded knowledge and call an external API through an action definition.

What you'll learn

- The roles of an API endpoint and an OpenAPI schema.
- How a GPT action converts a user request into an API call.
- Why authentication, billing, and ownership matter.

Step-by-step notes

1. **Start with an available API.** The examples use simple weather or dummy-JSON services to show that an application owner exposes endpoints for software access.
2. **Describe it with OpenAPI.** An OpenAPI 3.1 document defines servers, paths, methods, parameters, and response shapes so the GPT knows what can be called.
3. **Add the schema as an action.** Configure authentication if required, then test the action in preview with a simple request.
4. **Inspect the actual response.** Confirm that the GPT used the correct endpoint and parameters and did not invent missing fields.
5. **Separate service costs.** A ChatGPT subscription and OpenAI API usage are different products; third-party APIs may also have their own quotas or charges.

Key takeaways

- Actions let a Custom GPT retrieve live data or perform operations.
- The OpenAPI schema is the contract between the GPT and service.
- Test authentication, errors, and cost limits before sharing the GPT.

Class 17: Create, test, and publish a Custom GPT

Class video: Open on YouTube

This class walks through the Custom GPT builder and shows how instructions, knowledge, tools, and sharing settings work together.

What you'll learn

- How to define a GPT's role and boundaries.
- When to use instructions versus uploaded knowledge.
- How preview testing and publishing options affect users.

Step-by-step notes

1. **Define the job clearly.** State the audience, supported questions, desired response style, and what the GPT must refuse or redirect.
2. **Write behavioral instructions.** Tell the GPT how to answer, how to use sources, and how to handle unrelated questions. Long reference material should not be pasted into the instruction box.
3. **Use knowledge files for large content.** When reference material exceeds the builder's instruction limits, upload clean, searchable documents and tell the GPT when to consult them.
4. **Test in Preview.** Try an expected question, an ambiguous question, an out-of-scope request, and a question whose answer exists only in the uploaded material.
5. **Choose visibility deliberately.** Keep it private while testing, use link-only sharing for a controlled audience, or publish publicly only after checking name, description, icon, policies, and source rights.

Key takeaways

- Instructions control behavior; knowledge files supply reference content.
- Boundary tests are as important as successful-answer tests.
- Publishing changes the audience and risk, not just the URL.

Class 18: Build a multimedia plugin with Plugin Creator

Class video: Open on YouTube

This class packages image generation, Gemini text-to-speech, and FFmpeg assembly into a reusable video plugin.

What you'll learn

- How several capabilities become one user-facing plugin.
- How inputs such as duration and aspect ratio drive the workflow.
- How to package and export the result.

Step-by-step notes

1. **Specify the plugin contract.** Inputs include topic, target duration, aspect ratio, and language; the output is a playable video plus intermediate assets when useful.
2. **Compose existing capabilities.** Use image generation for visuals, Gemini TTS for narration, and FFmpeg for timing and encoding instead of rebuilding each component.
3. **Calculate scene count and timing.** Divide the target duration into sensible scene lengths and keep narration short enough to fit each scene.
4. **Store secrets outside the package.** The plugin may describe required environment variables, but it must not include a real API key in exported files.
5. **Validate and export.** Run a small example, inspect audio/video synchronization, then package the plugin as a ZIP for installation elsewhere.

Key takeaways

- Plugins are valuable when they hide a multi-tool production pipeline.
- Duration and scene timing must be calculated, not guessed.
- A distributable plugin should contain instructions and code, never credentials.

Class 19: Media-plugin practice and an MCP bridge

Class video: Open on YouTube

This class reinforces the custom media workflow and prepares the transition from local packaged capabilities to standardized MCP integrations.

What you'll learn

- How to retest a plugin with different inputs.
- How subscriptions and generation credits affect a workflow.
- Why MCP becomes relevant when capabilities live in external applications.

Step-by-step notes

1. **Repeat the workflow with a changed requirement.** A second topic, language, or aspect ratio reveals which values were accidentally hard-coded.
2. **Track quota-bearing steps.** Image, video, and speech generation may consume separate credits. Estimate the number of calls before a long run.
3. **Inspect intermediate outputs.** Keep the scene plan, prompts, images, and audio long enough to diagnose a bad final video.
4. **Distinguish local composition from service integration.** A plugin can coordinate a local workflow; an MCP server can expose external tools to compatible AI hosts through a standard interface.
5. **Prepare an integration checklist.** Identify the underlying API, authentication, allowed actions, required parameters, expected responses, and failure cases.

Key takeaways

- Repetition with varied inputs is the best test of reusability.
- Credit limits are part of technical design.
- MCP starts with a clear understanding of the service being exposed.

Class 20: MCP architecture through the restaurant analogy

Class video: Open on YouTube

This class explains MCP architecture with a restaurant: the guest asks, the waiter coordinates, the kitchen manager exposes available work, and cooks perform specialized tasks.

What you'll learn

- How host, client, server, tool, and underlying application relate.
- Why the model should not directly know every service implementation.
- How an MCP request and response move through the system.

Step-by-step notes

1. **Host = restaurant experience.** The AI application receives the user's request and provides the conversation interface.
2. **Client = waiter.** The MCP client maintains the connection, discovers available capabilities, and carries structured requests and results.
3. **Server = kitchen manager.** The MCP server advertises tools/resources and translates protocol calls into operations the underlying system understands.
4. **Application/API = cook and kitchen.** The real service performs the work—for example, obtaining weather data or querying a database.
5. **Follow one example.** “What is today's weather in Hyderabad?” becomes a structured weather-tool call; the service response returns through the server and client; the model explains it to the user.

Key takeaways

- Each layer has a distinct responsibility.
- The model selects capabilities; the server implements them.
- Structured tool results reduce ambiguity compared with copying web text.

Class 21: Wrap an existing API as an MCP server

Class video: Open on YouTube

This class shows that an MCP server commonly sits in front of an existing API and adapts it for AI-tool use.

What you'll learn

- Why APIs and MCP servers are complementary.
- How a tool schema maps to an API request.
- What the application owner must provide and maintain.

Step-by-step notes

1. **Keep the distinction clear.** An API enables software-to-software communication. MCP describes how an AI host discovers and invokes model-facing capabilities.
2. **Choose a narrow operation.** For example, `get_weather(city)` is easier to describe, authorize, and test than a single unrestricted "call anything" tool.
3. **Validate tool arguments.** Check required fields, permitted values, and lengths before calling the upstream API.
4. **Translate the response.** Return structured, relevant data and useful errors; do not leak raw credentials, internal stack traces, or unnecessary personal data.
5. **Treat the wrapper as production software.** The server needs versioning, authentication, logging, rate-limit handling, tests, and documentation.

Key takeaways

- MCP does not replace the underlying API.
- Small, well-described tools are easier for models to use correctly.
- The service owner remains responsible for security and reliability.

Class 22: Build a movie-booking MCP server

Class video: Open on YouTube

This class applies MCP concepts to a movie domain, where tools can search shows and work with cities, theatres, dates, or seats.

What you'll learn

- How to convert business operations into MCP tools.
- How JSON schemas make inputs unambiguous.
- Why read and write operations should be separated.

Step-by-step notes

1. **Model the domain.** Identify stable concepts such as movie, city, theatre, show, seat, price, and booking identifier.
2. **Create small tools.** Examples are `list_movies(city)`, `get_shows(movie, date)`, and `get_available_seats(show_id)`.
3. **Define JSON inputs and outputs.** Use clear types, descriptions, required fields, and enumerations where appropriate.
4. **Separate discovery from booking.** Searching is read-only; reserving or purchasing changes state and should require clearer confirmation and authentication.
5. **Test predictable failures.** Invalid city, past date, missing show, unavailable seat, upstream timeout, and duplicate booking should return actionable errors.

Key takeaways

- Domain modeling comes before tool coding.
- Structured schemas guide both the model and the server.
- State-changing actions need stronger safeguards than searches.

Class 23: Deploy an MCP-backed movie application

Class video: Open on YouTube

This class moves the movie example from local development to a hosted application and discusses deployment configuration.

What you'll learn

- What must change when a local app is deployed.
- How environment variables and public URLs are managed.
- How to test the deployed version independently.

Step-by-step notes

1. **Prepare a production build.** Remove local-only paths and make service URLs configurable.
2. **Deploy the web application.** A host such as Netlify can build and serve the frontend, but backend or MCP services may need a separate compatible runtime.
3. **Configure secrets on the host.** Add API keys through the hosting dashboard or secret store. Never commit a `.env` file containing live values.
4. **Update allowed origins and callback URLs.** Cross-origin rules, authentication callbacks, and MCP configuration must refer to the deployed addresses.
5. **Run a clean-browser test.** Open the public URL without local development services, execute the core flow, and inspect both browser and server logs if it fails.

Key takeaways

- “Works locally” does not prove a deployable architecture.
- Public URLs and secrets belong in environment-specific configuration.
- Test the live deployment without relying on the developer machine.

Class 24: MCP configuration and connecting the server to Codex

Class video: Open on YouTube

This class focuses on the configuration that tells an AI host how to start or reach an MCP server.

What you'll learn

- What an MCP configuration entry describes.
- The difference between local-process and remote-server connections.
- How to verify tool discovery before using a tool.

Step-by-step notes

1. **Choose the transport.** A local server may be launched as a command and communicate over standard input/output; a remote server uses a network URL and usually authentication.
2. **Add the server definition.** Configure a unique name, command or URL, arguments, and environment-variable references.
3. **Avoid literal secrets.** Point to environment variables or an approved credential mechanism rather than embedding tokens in shared JSON.
4. **Restart or reload the host.** Many clients discover MCP servers when a session starts, so configuration changes may not appear immediately.
5. **Verify discovery.** Confirm the server connects and that the expected tool names and descriptions appear before attempting a real task.

Key takeaways

- Configuration is the bridge between the host and server.
- Transport and authentication determine the required fields.
- Discovery checks isolate connection problems from tool-logic problems.

Class 25: Use the movie MCP workflow from another client

Class video: Open on YouTube

This class reinforces MCP portability by connecting the same movie capability from a client and completing a user-facing flow.

What you'll learn

- How a compatible client discovers an existing MCP server.
- How tool calls appear during a conversation.
- How to diagnose configuration, server, and application failures separately.

Step-by-step notes

1. **Reuse the server instead of rewriting the integration.** Add the server to the new client's MCP configuration using its supported transport.
2. **Start with a read-only request.** Ask for movies or shows in a city and inspect the proposed tool call and returned data.
3. **Check parameter grounding.** The client should pass the city, movie, date, or show identifier explicitly rather than inventing defaults.
4. **Confirm before a state-changing step.** Seat selection or booking should display the relevant details and obtain user confirmation.
5. **Debug by layer.** If tools are absent, check client configuration; if calls fail, check server logs; if data is wrong, check the upstream application/API.

Key takeaways

- Portability is a central benefit of MCP.
- Read-only tests are the safest first connection check.
- Layered debugging prevents guessing.

Class 26: Build an OpenAI-powered chatbot application

Class video: Open on YouTube

This class uses a developer plugin and the OpenAI API to create a chatbot-style application, run it locally, and prepare it for publishing.

What you'll learn

- How a frontend sends a request to an AI-enabled backend.
- How to create and use an OpenAI API key safely.
- What changes between a local demo and a published app.

Step-by-step notes

1. **Write the application brief.** Define the user, purpose, inputs, outputs, visual style, and the model-powered behavior.
2. **Create the API key in the OpenAI developer platform.** Store it only in a server-side environment variable. Browser JavaScript must never expose a secret key.
3. **Build the request path.** Browser → your backend endpoint → OpenAI API → backend response → browser display. The backend controls secrets, validation, and limits.
4. **Run locally and inspect errors.** Test missing input, network failure, invalid key, rate limits, and long responses—not just the happy path.
5. **Publish with production configuration.** Set the secret on the hosting platform and verify the deployed app. If an MCP App UI is involved, also test its host integration.

Key takeaways

- Keep API keys on the server.
- A useful AI app needs error, loading, and usage-limit behavior.
- Deployment requires new configuration and a fresh end-to-end test.

Class 27: Build an AI palm-reading app and a self-contained 3D game

Class video: Open on YouTube

This class demonstrates turning visual references and product ideas into working applications, then iterating on the generated result.

What you'll learn

- How a reference image can guide layout and visual direction.
- How to specify an AI feature separately from the interface.
- When a single-file app is useful for a prototype.

Step-by-step notes

1. **Describe the product flow.** For a palm-reading example: upload or capture an image → validate it → send the appropriate data to a model-backed service → present a clearly labeled entertainment-style result.
2. **Use references for design, not blind copying.** Extract layout, spacing, color, and interaction ideas while avoiding protected branding or misleading claims.
3. **Protect user data.** Images can be sensitive. Explain retention, minimize uploads, and avoid presenting generated interpretations as medical or factual diagnosis.
4. **Prototype a self-contained game.** Inline HTML, CSS, and JavaScript can make a portable demo, but larger projects should be split into maintainable modules.
5. **Iterate through observed defects.** Test controls, collisions, scoring, restart behavior, mobile layout, and performance, then give the agent concrete fixes.

Key takeaways

- A reference is most useful when translated into explicit design requirements.
- AI features require privacy and expectation-setting.
- Single-file prototypes are convenient, not automatically production-ready.

Class 28: Codex cloud tasks, uploaded skills, and scheduled work

Class video: Open on YouTube

This class contrasts desktop work with cloud-executed tasks and demonstrates a recurring workflow such as producing cricket-news audio and sending it through a messaging channel.

What you'll learn

- When a cloud task is more suitable than an interactive desktop session.
- How an uploaded skill standardizes recurring output.
- What a reliable schedule must specify.

Step-by-step notes

1. **Choose the execution environment.** Use the desktop for interactive work with local files and applications; use a cloud task for work that must run when the local machine is unavailable.
2. **Upload the reusable workflow.** A skill can define how to gather news, verify recency, write the script, synthesize audio, and format the outgoing message.
3. **Specify the schedule completely.** Include frequency, exact local time, timezone, start conditions, and what to do after a missed run.
4. **Separate creation from delivery.** Generate and validate the audio first; then send it through the authorized channel. This makes failures easier to diagnose.
5. **Test once before recurrence.** Run the complete workflow manually, inspect the sources and audio, then enable the recurring schedule.

Key takeaways

- Cloud execution supports unattended recurring work.
- Timezone and delivery behavior are part of the specification.
- A scheduled mistake repeats automatically, so test the full run first.

Class 29: Why run AI models locally?

Class video: Open on YouTube

This class introduces open models and explains the privacy, cost, connectivity, and hardware trade-offs of local inference.

What you'll learn

- The practical differences between local and hosted models.
- Why model size and hardware requirements matter.
- Which workloads benefit from local execution.

Step-by-step notes

1. **Understand the local path.** The model files and inference software run on your own machine, so prompts do not need to be sent to a model provider's cloud.
2. **Evaluate benefits.** Local models can provide offline access, better control over data, and no per-request API charge after hardware and electricity costs.
3. **Evaluate limitations.** Model downloads are large, RAM/VRAM requirements can be significant, inference may be slower, and local models may be less capable than leading hosted models.
4. **Match model to hardware.** Check parameter count, quantization, context size, and recommended memory. A smaller quantized model is often a better starting point.
5. **Choose by workload.** Local execution is attractive for private experimentation and predictable repetitive tasks; hosted models remain useful for maximum capability and minimal setup.

Key takeaways

- Local does not automatically mean free or faster.
- Hardware capacity determines the practical model choices.
- Use privacy, quality, latency, and cost together when deciding.

Class 30: Install and use Ollama

Class video: Open on YouTube

This class presents Ollama as the runtime that downloads and serves local models; Ollama itself is not the model.

What you'll learn

- How Ollama relates to model files.
- The basic pull, run, list, and remove workflow.
- How to compare local and hosted inference responsibly.

Step-by-step notes

1. **Install the runtime for your operating system.** Confirm the service starts and the `ollama` command is available.
2. **Download a model.** Use `ollama pull <model>` or run it directly with `ollama run <model>`, which downloads missing files first.
3. **Interact from the terminal.** Test a short prompt, then inspect speed, memory use, response quality, and context handling.
4. **Manage local models.** `ollama list` shows installed models; remove an unused model only after confirming its exact name because model files can be large.
5. **Connect applications through the local service.** Tools such as Open WebUI or agents can call Ollama's local endpoint, subject to network and firewall configuration.

Key takeaways

- Ollama is a model runner and server.
- Start with a model that fits available memory.
- Local endpoints should not be exposed publicly without authentication and network controls.

Class 31: Run Ollama on a cloud GPU virtual machine

Class video: Open on YouTube

This class moves local-model concepts to a rented GPU machine and explains why remote compute can help when a laptop lacks sufficient hardware.

What you'll learn

- What a GPU VM instance provides.
- How to estimate cost and choose hardware.
- How remote model access changes the security model.

Step-by-step notes

1. **Select a suitable instance.** Consider GPU memory, system RAM, storage, operating system image, hourly price, region, and expected availability.
2. **Remember that a VM is still a computer.** Required software—drivers, Ollama, Docker, and user interfaces—must already be in the chosen image or installed afterward.
3. **Connect securely.** Use SSH with a key pair and a non-root workflow where possible. Keep the private key only on trusted devices.
4. **Install and test Ollama remotely.** Pull a small model first and confirm inference before downloading a larger one.
5. **Control cost and exposure.** Stop or delete idle instances, restrict firewall ports, and avoid exposing the Ollama service directly to the public internet.

Key takeaways

- Cloud GPUs trade setup and hourly cost for stronger hardware.
- Images are reusable software templates, not ordinary pictures.
- Shutdown, firewall, and key management are essential operating tasks.

Class 32: Cloud images, SSH keys, and secure VM access

Class video: Open on YouTube

This class deepens the cloud setup by explaining reusable machine images and public/private keys for SSH access.

What you'll learn

- How a cloud image prepackages an operating system and software.
- The distinct purposes of public and private SSH keys.
- How to connect to a VM without sharing passwords.

Step-by-step notes

1. **Choose an image as a starting template.** An image may include Linux plus tools such as Python, Docker, or GPU drivers. You can still install additional software after launch.
2. **Create an SSH key pair.** The public key may be installed on the server; the private key remains with the user and proves identity.
3. **Protect the private key.** Do not email it, upload it to source control, or place it in a shared folder. If it is exposed, replace the key.
4. **Connect with the key.** A typical form is `ssh -i <private-key-file> <username>@<server-ip>`. The exact username depends on the selected image.
5. **Troubleshoot systematically.** Check instance state, public IP, firewall port 22, username, key path, and file permissions before changing unrelated settings.

Key takeaways

- A machine image accelerates repeatable VM setup.
- Public keys are shared with servers; private keys must remain secret.
- Most SSH failures can be isolated with a short connectivity checklist.

Class 33: Open WebUI, Docker, and remote Ollama access

Class video: Open on YouTube

This class adds a browser interface to Ollama on a Linux VM and explains the network path between Docker, the host, and the model service.

What you'll learn

- Why Open WebUI is separate from Ollama.
- How Docker packages the web interface.
- How ports and host addresses affect connectivity.

Step-by-step notes

1. **Assign each component a role.** Ollama serves the model; Open WebUI provides chat history, model selection, and a browser interface; Docker runs Open WebUI in an isolated container.
2. **Start the container with persistent storage.** Map the web port and a named volume so configuration and conversations are not lost when the container is replaced.
3. **Point the container to Ollama.** `localhost` inside the container refers to that container, not automatically to the Linux host. Use the platform-supported host address or Docker networking configuration.
4. **Open only the required port.** Restrict the cloud firewall to your IP during testing. A public UI should have authentication and HTTPS.
5. **Test the complete path.** Browser → Open WebUI container → Ollama service → selected model. Check container logs and Ollama status at the broken layer.

Key takeaways

- Open WebUI is a client interface; Ollama performs inference.
- Container networking is the most common connection trap.
- Do not expose an unauthenticated model UI to the internet.

Class 34: Install Hermes Agent on Windows

Class video: Open on YouTube

This class begins the Hermes Agent section with prerequisites, installation, and practical troubleshooting on Windows.

What you'll learn

- What an agent harness adds around an AI model.
- Which development tools commonly need to be installed first.
- How to diagnose installation failures from logs.

Step-by-step notes

1. **Understand the product category.** Hermes Agent is an agentic environment that can combine a model with tools, skills, memory, and task execution.
2. **Check prerequisites.** The session reviews dependencies such as Python, Git, and Node.js. Confirm each from a new terminal so PATH changes are recognized.
3. **Follow the official installer or repository instructions.** Use the documented version requirements rather than mixing commands from old tutorials.
4. **Read the first meaningful error.** If setup fails, capture the exact command, exit code, and earliest relevant error instead of repeatedly reinstalling everything.
5. **Run a minimal smoke test.** Start the agent, select a basic configuration, and ask a harmless question before adding models, skills, or integrations.

Key takeaways

- The harness is more than the model it uses.
- Version and PATH problems cause many Windows setup failures.
- A minimal successful start is the foundation for later customization.

Class 35: Use different cloud and local models in Hermes Agent

Class video: Open on YouTube

This class explores model-provider flexibility: an agent harness can use different “brains” while keeping its surrounding tools and workflow.

What you'll learn

- Why model choice is separate from the agent platform.
- How API-based providers differ from local endpoints.
- How to choose between free, paid, and self-hosted options.

Step-by-step notes

1. **Separate harness from model.** Hermes supplies the agent environment; OpenAI, DeepSeek, OpenRouter, or a local Ollama model may supply inference.
2. **Add one provider at a time.** Configure the base URL when required, add the API key through a secure setting, and select a supported model identifier.
3. **Test with the same prompt.** Compare response quality, latency, tool use, context limits, and cost on a small repeatable task.
4. **Understand “free.”** A free tier may have rate limits, lower availability, data-use conditions, or model restrictions. A local model avoids request fees but consumes hardware.
5. **Keep a fallback.** If the primary provider is unavailable or unsuitable, a second configured model can keep the workflow usable.

Key takeaways

- Harness capabilities can remain stable while models change.
- Base URL, model ID, and API key must match the same provider.
- Select models by task quality and operating constraints, not label alone.

Class 36: Configure providers, endpoints, and skills in Hermes

Class video: Open on YouTube

This class goes deeper into provider configuration and shows how skills extend the agent after a model is connected.

What you'll learn

- What a base URL or endpoint means.
- How provider configuration affects model discovery.
- How skills add repeatable behavior to the harness.

Step-by-step notes

1. **Use the provider's documented endpoint.** A base URL tells the client where compatible API requests should be sent. A valid key for a different endpoint will not work.
2. **Select an exact model identifier.** Display names and API IDs can differ. Copy the supported ID from the provider's current documentation or model list.
3. **Test credentials without exposing them.** Confirm a small request succeeds, then redact keys from screenshots, logs, and shared configuration.
4. **Add skills for task-specific behavior.** A skill can teach the agent a workflow such as research, coding, or media generation while the configured model performs the reasoning.
5. **Diagnose by configuration layer.** Authentication errors point to credentials; “model not found” points to the ID or endpoint; tool failures point to the harness or skill.

Key takeaways

- Endpoint, credential, and model ID form one configuration unit.
- Skills extend behavior without replacing the model.
- Error classification speeds up troubleshooting.

Class 37: Prompt, context, harness, and loop engineering

Class video: Open on YouTube

This class compares four levels of designing AI work, from a single instruction to a self-correcting system.

What you'll learn

- The responsibility of each engineering layer.
- Why better prompts cannot fix missing tools or missing context.
- How feedback loops improve reliability.

Step-by-step notes

1. **Prompt engineering:** improve the immediate instruction—goal, audience, constraints, format, examples, and success criteria.
2. **Context engineering:** provide the right files, conversation history, retrieved knowledge, tool outputs, and state while excluding distracting material.
3. **Harness engineering:** design the environment around the model—tools, skills, memory, permissions, storage, subagents, and scheduling.
4. **Loop engineering:** create an iterative cycle such as plan → execute → test → review → revise → stop when acceptance criteria pass.
5. **Diagnose at the correct layer.** If the model lacks a file, fix context; if it cannot act, add a tool; if errors persist unchecked, add a review loop.

Key takeaways

- A prompt is only one layer of an agentic system.
- Context determines what the model can ground its answer in.
- Loops turn one-shot generation into verified work.

Class 38: Connect agent harnesses to local or cloud models

Class video: Open on YouTube

This class combines earlier concepts by connecting an agentic platform to models running locally, on a remote VM, or through a hosted API.

What you'll learn

- How deployment location changes configuration.
- How to expose a remote model service safely.
- How cost and performance affect the choice.

Step-by-step notes

1. **Map the topology.** Identify where the harness runs, where the model runs, and which network connection joins them.
2. **Use the correct endpoint.** A local harness may call local Ollama directly; a harness on another machine needs a reachable private address, tunnel, or protected endpoint.
3. **Restrict access.** Prefer private networking or SSH tunneling. If a port must be reachable, use firewall allow-lists and authentication.
4. **Compare operational trade-offs.** Local models offer control; cloud GPU VMs add hourly cost and maintenance; hosted APIs simplify infrastructure but charge by usage.
5. **Run an agent-level test.** Verify not only chat completion but also context length, tool calls, cancellation, and recovery from provider errors.

Key takeaways

- Model location changes networking and security requirements.
- A reachable endpoint is not necessarily a safe endpoint.
- Test the model through the actual agent workflow.

Class 39: Connect a Telegram bot to an AI agent

Class video: Open on YouTube

This class uses Telegram as a messaging channel so users can interact with an agent outside its local interface.

What you'll learn

- How a bot token links Telegram to the agent.
- How messages travel between the user, bot, harness, and model.
- What access controls a messaging bot needs.

Step-by-step notes

1. **Create the bot through Telegram's official bot-management flow.** Record the bot username and token; treat the token like a password.
2. **Configure the agent channel.** Add the token through a secret setting or environment variable and enable the Telegram connector.
3. **Test with one account.** Send a simple message and confirm the reply comes from the intended model and agent configuration.
4. **Restrict who can use it.** Use allowed chat/user IDs, private groups, or an approval mechanism. An unrestricted bot can consume resources or expose tools.
5. **Handle channel limitations.** Messages have formatting and size constraints; long outputs, files, and failures need clear fallback behavior.

Key takeaways

- The bot token grants control of the bot and must remain secret.
- Messaging is a channel; the harness and model still do the work.
- Authorization matters even when the bot is not publicly advertised.

Class 40: Schedule recurring agent jobs

Class video: Open on YouTube

This class demonstrates a scheduled task—for example, sending a psychology quote with a short explanation every day.

What you'll learn

- How a natural-language schedule maps to a cron-style job.
- What information a reliable recurring task needs.
- How to observe, edit, and stop scheduled work.

Step-by-step notes

1. **Define the payload.** Specify exactly what should be created or sent, its length and tone, and the destination.
2. **Define time precisely.** “Every day at 11:45 p.m.” is incomplete without a timezone. Also decide what happens during daylight-saving changes or missed runs.
3. **Create the job and inspect its saved definition.** Confirm the next-run time, recurrence, enabled status, and associated agent/model.
4. **Test with a near-term one-off run.** This validates content creation and delivery without waiting a full day.
5. **Monitor and maintain.** Check execution history, failures, token/API usage, and stale credentials. Disable or delete jobs that are no longer needed.

Key takeaways

- Scheduling automates repetition, including repeated mistakes.
- Timezone, destination, and failure handling belong in the requirement.
- Logs and a stop mechanism are mandatory operational features.

Class 41: Agent harnesses and complete project execution

Class video: Open on YouTube

This class explains how a harness lets an agent inspect a project, modify files, run commands, test the result, and report completion.

What you'll learn

- What makes an agentic coding task different from a chat answer.
- How tools and workspace context support end-to-end execution.
- How to write a completion-oriented request.

Step-by-step notes

1. **Ask for an outcome, not just code text.** A useful request is: inspect the project, identify the defect, modify the correct files, run tests, and provide the verified result.
2. **Let the harness expose controlled tools.** File access, terminal commands, browser checks, memory, and skills give the model ways to gather evidence and act.
3. **Set boundaries.** Identify the workspace, allowed systems, prohibited changes, and whether external publication is in scope.
4. **Require a verification loop.** The agent should run the application or tests, inspect failures, revise, and repeat until the stated criteria pass.
5. **Review the handoff.** Expect changed files, test evidence, known limitations, and a direct artifact or preview—not only “completed successfully.”

Key takeaways

- A harness turns model output into controlled project work.
- Clear scope prevents unnecessary changes.
- Evidence-based completion is more trustworthy than a generated explanation.

Class 42: Troubleshoot a harness and choose models by context limits

Class video: Open on YouTube

This class combines practical troubleshooting with model-selection concepts such as context windows and token usage.

What you'll learn

- How to capture a reproducible UI or workspace problem.
- What tokens and context windows mean operationally.
- Why a larger context window is not the only model criterion.

Step-by-step notes

1. **Record the failing behavior.** Capture the action, observed result, expected result, error text, and environment. A short screen recording can be useful for an intermittent UI problem.
2. **Restart the correct service.** If a harness server is stuck, use its documented stop/start commands and verify the process and port rather than rebooting everything blindly.
3. **Understand context capacity.** The context window includes instructions, conversation, retrieved files, and outputs. Large projects can crowd out earlier information.
4. **Control token use.** Supply relevant files, summarize old material, and avoid repeated logs. More context can increase latency and cost and can distract the model.
5. **Choose the model for the task.** Balance reasoning quality, coding/tool reliability, context size, speed, cost, availability, and privacy.

Key takeaways

- Reproducible evidence makes troubleshooting faster.
- Context is a limited working space, not permanent memory.
- Model selection is a multi-factor engineering decision.

Class 43: Observe LLM applications with Langfuse

Class video: Open on YouTube

This class introduces Langfuse as an observability platform for understanding what happens inside an LLM application.

What you'll learn

- Why ordinary uptime monitoring is not enough for AI workflows.
- The meaning of traces, spans, and generations.
- Which quality, latency, token, and cost signals to inspect.

Step-by-step notes

1. **Treat observability as structured evidence.** A successful HTTP response can still contain a poor, expensive, or incorrectly grounded model answer.
2. **Use a trace for one end-to-end request.** It links retrieval, tool calls, model generations, and post-processing into a single timeline.
3. **Use spans for individual operations.** A retrieval query, function call, or parsing step can be timed and annotated separately.
4. **Record generations responsibly.** Capture model, prompt version, latency, token counts, cost, and evaluation scores while redacting sensitive content.
5. **Turn findings into action.** Locate slow steps, repeated calls, prompt regressions, failed tools, or unexpectedly expensive requests and then compare after a fix.

Key takeaways

- Langfuse helps explain behavior, not just uptime.
- Traces show the full request; spans show its parts.
- Observability data must follow privacy and retention rules.

Class 44: Install and configure OpenClaw

Class video: Open on YouTube

This class introduces OpenClaw as an open-source agent harness, installs its web interface, selects a model provider, and performs a first task.

What you'll learn

- What a harness supplies around a borrowed model.
- How to install OpenClaw and complete first-time setup.
- How channels, search providers, and models fit into the configuration.

Step-by-step notes

1. **Install the package using the current official command.** The class demonstrates a global npm installation; verify the present package name and documentation before running it.
2. **Complete onboarding.** Choose a setup mode, model provider, authentication method, default model, optional messaging channel, and search provider.
3. **Use a secret for the API key.** A selected OpenAI model provides the “brain”; OpenClaw supplies the agent infrastructure around it.
4. **Start the gateway and web UI.** If the page cannot connect, inspect the gateway process, local port, logs, and any password or token required by the UI.
5. **Run a small creation task.** Build and test a simple calculator or other self-contained app, then inspect the generated files and behavior.

Key takeaways

- The harness and the underlying model are separate components.
- First-time onboarding establishes only a starting configuration.
- Installation instructions can change, so use the current official source.

Class 45: Add a Gemini TTS skill to OpenClaw

Class video: Open on YouTube

This class extends OpenClaw with a speech-generation skill and resolves the environment configuration needed to run it.

What you'll learn

- How a skill adds a new media workflow to an agent harness.
- Why environment variables are used for API credentials.
- How to test multilingual narration or song output.

Step-by-step notes

1. **Install or add the TTS skill.** Read its prerequisites, supported languages, voice parameters, and expected output format.
2. **Set the required environment variable.** Use the operating system or harness secret configuration so the Gemini key is available to the process that runs the skill.
3. **Restart after configuration changes.** A running gateway may not inherit environment variables added later.
4. **Test a short multilingual sample.** The class uses Telugu-oriented content such as a movie review or song. Check pronunciation, script, pace, and audio quality before generating a long file.
5. **Keep content generation modular.** First prepare the script, then synthesize audio, then save or deliver it. This makes corrections cheaper and easier.

Key takeaways

- Skills can add production workflows without changing the harness itself.
- Environment variables must reach the actual running process.
- Validate text before paying the cost of full audio generation.

Class 46: Multi-agent orchestration and loop engineering

Class video: Open on YouTube

This class concludes with multiple specialized agents working in parallel or sequence—for example, planner, developer, reviewer, and auditor.

What you'll learn

- Why specialized subagents can improve a large task.
- How independent context windows protect the main conversation.
- How review loops catch defects before final handoff.

Step-by-step notes

1. **Split work by responsibility.** A planner defines tasks, developers implement bounded parts, a reviewer checks correctness, and an auditor verifies requirements and risk.
2. **Delegate only independent work.** Parallelism helps when tasks do not edit the same files or depend on unfinished results. Overlapping ownership creates conflicts.
3. **Give every subagent a concrete contract.** State inputs, output format, boundaries, relevant files, and completion criteria.
4. **Use separate context windows.** Background agents can examine detailed logs or files without filling the main context; they should return concise evidence and conclusions.
5. **Close the loop.** Integrate results, run the full test suite, send defects back to the appropriate worker, and stop only when the shared acceptance criteria pass.

Key takeaways

- More agents do not automatically mean better results.
- Clear ownership and independent tasks make parallel work effective.
- A separate reviewer often catches assumptions the implementer missed.

Course-wide quick reference

Choose the right extension

Need	Best fit
Reusable instructions, examples, or workflow	Skill
Packaged capabilities and integrations	Plugin
Standard connection from an AI host to external tools/data	MCP server
Repository-specific commands and rules	`AGENTS.md`
Tools, memory, scheduling, and execution around a model	Agent harness

Reliable agent loop

1. Define the outcome, scope, constraints, and acceptance criteria.
2. Provide only the relevant context and authorized tools.
3. Execute in small, observable steps.
4. Test or render the actual output.
5. Review failures, revise, and repeat.
6. Report evidence, limitations, and the final artifact.

Security checklist

- Keep API keys, bot tokens, and SSH private keys out of source files and screenshots.
- Inspect third-party skills, plugins, MCP servers, and install commands before using them.
- Confirm before state-changing actions such as booking, sending, publishing, or deleting.
- Restrict public ports, use authentication, and prefer private networking or tunnels.
- Monitor recurring jobs, cloud costs, token usage, and stale credentials.

Final revision questions

- Can you explain the difference between a skill, plugin, MCP server, and agent harness?
- Can you write a task with clear scope, constraints, acceptance criteria, and a verification step?
- Can you trace an MCP request from the user to the underlying application and back?
- Can you keep API keys, bot tokens, and SSH private keys out of code and shared files?
- Can you compare local models, cloud GPU models, and hosted APIs using privacy, cost, latency, and quality?
- Can you design a plan → execute → test → review → revise loop for a real project?